

A Model Driven Agent-Oriented Self-Adaptive Software Development Method

Yiwei Lei, Kerong Ben, Zhiyong He

Department of Computer Engineering
Naval University of Engineering
Hubei, Wuhan 430033
syjdlyw@163.com

Abstract—Software running in dynamic open environments needs to be able to autonomously adapt to changing contexts, which challenges traditional software development methods. Based on the existing work, we present a model-driven agent-oriented adaptive software development method. During the requirement analysis phase, we extend the Tropos goal model with external context conditions and internal event conditions, which can be utilized to specify adaptive software variations. In the design phase, we present an architecture design method based on model transformations, and provide a solution to map the requirement model to the design model. During the implementation phase, we design an integrated development environment which combines the Jadex platform and the JADE middleware, and propose a detailed implementation process based on the integrated environment.

Keywords—model-driven development; self-adaptive software; BDI agent; model transformation

I. INTRODUCTION

Self-adaptive applications can autonomously adapt themselves to changing environments without manual interventions, which can reduce the maintenance cost and improve the software dependability. Due to increasing software complexities and environment dynamics, the adaptive ability of the software will become more important. Since the adaptive software will run in changing environments, developers need to predict the probable external context conditions and internal event conditions of the software to be, and to specify requirement variations in the requirement phase. Flexibility is a key criterion to evaluate the ability of the software to adapt to context variations, which demand developers to design an open and scalable architecture. Furthermore, the adaptive software development language and environment determines the development efficiency, software portability and the interoperability. Therefore, how to select an appropriate way to develop the adaptive software is a problem to address.

During the requirements analysis phase, object-oriented requirement modeling (OORM) [1] is used as a traditional requirement modeling technique. OORM extends the UML class diagram by adding context condition classes to enable the UML model to specify context variations. Component-based requirement modeling techniques [2] use self-description

components to model adaptive software requirements, which can be converted into runtime adaption strategies. Goal-oriented requirement modeling method [3] [4] [5] defines a set of graphical model notations to facilitate the communication between stakeholders, and allow users to participate in the process of adaption strategies making. However, the GORM is not able to model the adaptive software requirements for its lack of ability to specify context conditions. Context-extended GORM [6] [7] [8] incorporates context conditions into the goal model to enable the goal model to specify contexts. However, the context-extended goal model is not capable of modeling self-adaptive fault tolerant requirements for its lack of software internal event conditions.

In the design phase, software product line (SPL) [9] techniques are used to customize different adaptive software to cope with context variations. However, it is sometimes difficult to modify adaption strategies once the software deployed. Policy-based self-adaption model (PobSAM) [10] can formally model the adaption manager, the managed software and interactions between them. Besides, the PobSAM supports the model-driven development of adaptive software. Adaptive software based on feedback loops [11] adjusts the input according to the distinction between expected and actual output. All the above methods adopt a centralized control pattern which has to know the entire states of the system. However, due to the increasing complexity of the system, it is sometimes difficult to obtain a global state.

In this paper, we propose a model-driven agent-oriented adaptive software development method to address the problem mentioned above. Model-driven development method bridges the gap between the requirement model and the design model, which enable an autonomic transformation from the former to the latter. Agent-oriented technology employs autonomic and cooperative agents to realize the decentralized control mode, which fascinates the development of open and complex distributed adaptive systems. In order to specify adaptive software requirements, we extend the original Tropos goal model with context conditions and internal event conditions. In order to transform the requirement model into the design model without losing useful information, we extend the BDI (beliefs, desires, and intentions) model according to the extended Tropos goal model, and provide a solution to map the

This research is supported by the State Key Laboratory of Software Engineering (SKLSE 2012-09-38)

requirement model to the design model. During the design phase, we integrated the Jadex into the JADE platform, and proposed an implementation method based on the integrated platform.

II. GOAL ORIENTED REQUIREMENT ANALYSIS

Goal oriented requirement analysis is a mainstream technique in the requirement engineering, provides means to elicit, refine and specify requirements of the software. Tropos is one of the most popular requirement modeling methods. However, they are not able to model the adaptive software for their lack of ability to describe external context conditions and internal event conditions. In this paper, we extend the Tropos goal model by incorporating context conditions and internal event conditions, which enable the extended model to modeling the adaptive software requirements.

A. Tropos goal model

Tropos goal model [12] is one of the dominant methods in the goal oriented requirement engineering. Goals are a useful abstraction to represent the needs and expectations of stakeholders, and they offer a very intuitive way to elicit and analyze requirements. Tropos defines a series of graphic symbols to specify user requirements. The basic elements of Tropos goal model are presented as following.

Actor: the actor is a behavioral entity performing actions to achieve its goals.

Goal: the goal represents a function property expected by the stakeholders.

Soft goal: the soft goal represents a performance property expected by the stakeholders.

Task: the task represents an action performed by an actor to satisfy its goal.

Resource: the resource denotes messages and data passed between actors.

Goal decomposition: the goal decomposition is used to refine software requirements through decomposing a goal into sub-goals. Goal *decomposition* divide into *and decomposition* and *or decomposition*.

Means-end: the means-end is used to relate a leaf goal with tasks through refining the leaf goal into tasks.

Contribution link: the contribution link denotes the impact of the achievement of a goal on some soft goals. The contribution link divides into *positive contribution link* and *negative contribution link*.

Dependence link: the dependence link indicates that an actor depends on other actors to perform tasks to help achieve its goals.

B. Extended Tropos goal model

Tropos goal models lack elements for specifying context conditions and internal event conditions (such as software failures), thus, it is not capable of specifying adaptive software

requirements. Therefore, we extend the goal model with new goal types, context conditions, internal event conditions, etc.

Goal types extending: we extend the *goal* into the *perform goal*, *achieve goal*, and the *maintain goal*. The *perform goal* is not mandatory to be achieved, and it only needs to perform tasks attached to it. The *Achieve goal* defines a set of target states that must be reached, but it does not specify particular tasks to execute. Once the *maintain goal* achieved, it needs to stay in the achieved goal state until the maintain condition is invalid.

Goal conditions extending: in order to extend the goal conditions, we associate the *perform goal* with *context conditions*, associate the *achieve goal* with *context conditions*, *create conditions*, *drop conditions*, *failure conditions* and *target conditions*, and associate the *maintain goal* with *target conditions* and *maintain conditions*. The *context condition* determines when a goal to be adopted and is allowed to be empty (i.e., always true). The *create condition* determines when a goal is created. The *drop condition* determines when a goal will no longer need to be considered. The *failure condition* determines when the goal is considered failed. The *target condition* determines when the goal is satisfied. The *maintain condition* determines how to reach the state specified by the maintain goal, and it will keep in that state until the maintain condition is invalid.

Fig. 1 shows the meta-model of the extended Tropos goal model, in which the newly added elements are represented as classes in gray.

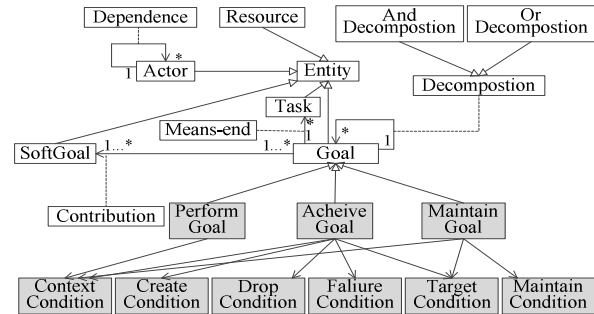


Figure 1. The meta-model of the extended Tropos goal models

C. An application example of the extended model

Fig. 2 shows an application example of the Ship Fire Damage Control System (SFDCS) to illustrate the extended Tropos goal model. Workflow of the SFDCS consists of the following three steps:

Step 1: Fire sensors analyze and reason on the environment data collected to determine whether a fire breaks out. When a fire breaks out, the fire situation (such as the type of the fire, the smoke concentration, and the spread speed of the fire) will be identified. Afterwards, the results will be updated to the SFDCS.

Step 2: The SFDCS analyzes and reasons based on the data updated by sensors, makes a decision how to delegate sub-goals to fire extinguishers according to their abilities.

Step 3: The fire extinguishers collaborate with each other to achieve the global goal of fire damage control through fulfilling obligations delegated to them.

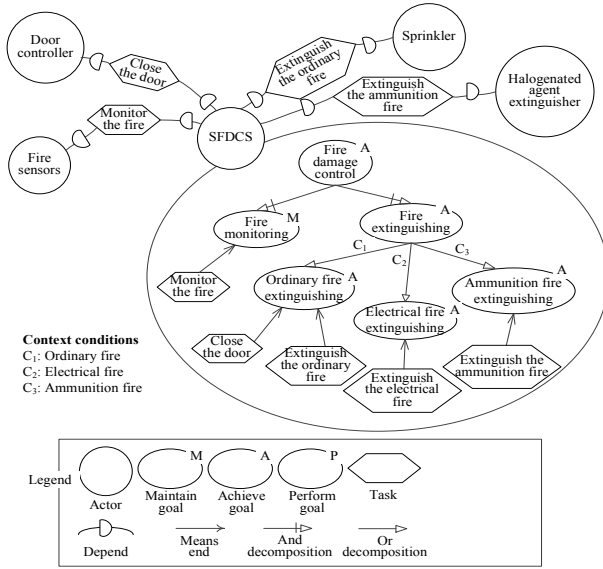


Figure 2. The extended Tropos goal model of the application example

III. AGENT ORIENTED SYSTEM DESIGN

Multi-agent systems are considered to be a new type of system architectures, which are widely used in large-scale complex adaptive system development. Multi-agent systems are self-organized, and they can dynamically reorganize themselves to adapt to the changing environment. The agent is essentially a particular software component that having the property of autonomy, self-introspection, sociality and mobility, etc. Agent can sense context changes, and adapt to changing contexts automatically by adjusting its behaviors. BDI (beliefs, desires and intentions) [13] is one of the most used agent models, and there are many ways to implement the BDI agent. The architecture of the BDI agent (As shown in Fig. 3) mainly consists of the following three components:

Belief: the belief represents the context information, the possible internal states, and the actions which can be performed by the agent.

Goal: the goal represents the agent intentions and the executable actions corresponding to each of intentions.

Plan: the plan represents a possible set of actions which can be performed by the agent to achieve a goal.

Tropos goal model has a set of notations familiar with the BDI agent model. Therefore, we can use a model transformation method to generate the BDI model from Tropos goal model. However, there are certain elements contained in the Tropos goal model but not included in the BDI model, such as the *dependence link*, *contribution link*, and *decomposition* and *or decomposition*. Thus, it is required to extend the BDI model referring to the Tropos goal model to avoid the loss of useful information during the transformation. The extended

meta-model of the BDI agent shows in Fig. 3, the newly added model elements are denoted as classes in gray.

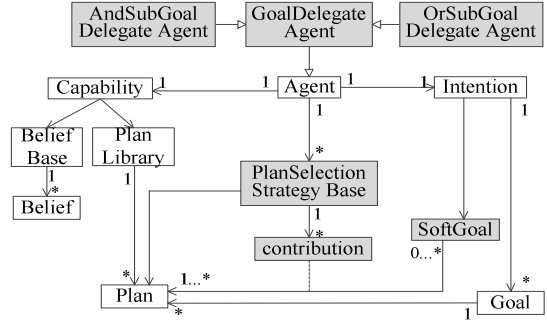


Figure 3. The meta-model of the extended BDI models

A. The transformation of the AND decomposition

Decomposition is used to refine software requirements through decomposing a goal into several sub-goals, which divide into *AND decomposition* and *OR decomposition*. In order to transform the decomposition into BDI models, we derive a *GoalDelegation* agent from the BDI agent. For *AND decomposition*, the father goal is satisfied only when all of its sub-goals are achieved. In order to enable the BDI model to describe the *AND decomposition*, we derive an *ANDDelegation agent* from the *GoalDelegation agent* (As shown in Fig. 3), which is in charge of delegating all the sub-goals to other agents to achieve. Thus, based on the extended BDI model, we can transform the *AND decomposition* into the organization relationship of agents. Fig. 4 shows a transformation from the goal *Fire damage control* to the BDI agent model.

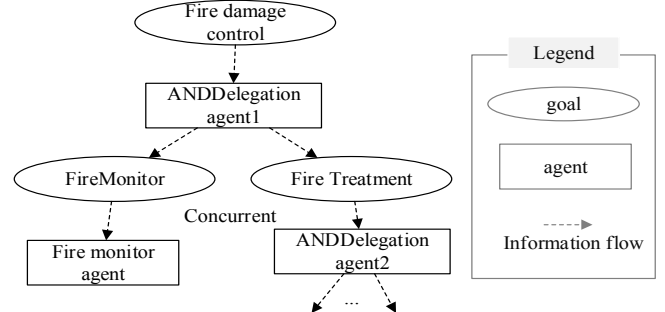


Figure 4. The transformation of AND decomposition

B. The transformation of the OR decomposition

For *OR decomposition*, as long as one of the sub-goals is met, the father-goal is achieved. In order to describe the *OR-decomposition*, we derive the *GoalDelegation* agent into the *ORDelegation agent* (As shown in Fig. 3). *ORDelegation agent* delegates one of the sub-goals to other agents to achieve. Thus, based on the extended BDI model, we can transform the *OR-decomposition* into the organization relationship of agents. Fig. 5 shows a transformation from the goal *Fire extinguishing* to the BDI agent model.

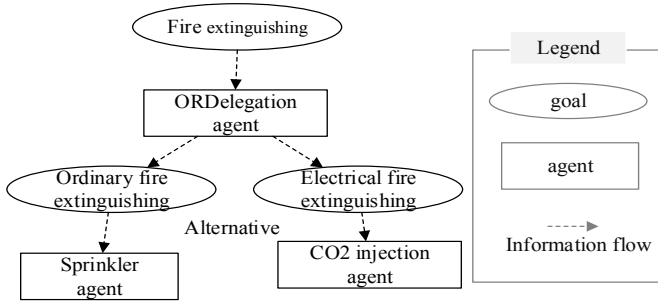


Figure 5. The transformation of OR decomposition

C. The transformation of the means-end

In the Tropos goal model, the *means-end* is utilized to refine the leaf goal into tasks, and the leaf goal is achieved by performing the tasks. However, a leaf goal can be refined into multiple tasks, so it is required to select one of the most appropriate plans to perform. The *contribution link* in Tropos represents influences of a plan to performance properties of the system, which can be used as a plan selection criterion. In order to model plan selection strategies, we extend the BDI model through incorporating an additional component called *plan selection strategy base* (As shown in Fig. 3). Based on the extended BDI model, we can transform the *means-end* into an achievement relationship between goals and plans of the agent, and transform the *contribution* into the utility in the BDI model.

D. The transformation of the dependency

Dependency in Tropos model indicates that the actors cannot achieve the goal by themselves, and need the assistance of the others. The actor depends on the other actors in terms of goal and plan. Goal or plan dependency indicates that the relationship between the actors are cooperative, thus, we can transform the dependency into the cooperative relationship between agents.

IV. SYSTEM IMPLEMENTATION BASED ON JADEX

Jadex is a reasoning machine and a development platform of the BDI agent, which can run upon the JADE middleware and used as an extension to the JADE. The middleware JADE provides the agent with basic services, which include the directory service, communication service and the management service, etc. Furthermore, JADE provides a suite of development and debug tools for the development of agents. The integrated architecture of the Jadex and JADE shows in Fig. 6. The Jadex agent is encapsulated in a JADE agent, which enables the Jadex agent to access basic services provided by the JADE, and to interact with each other.

Jadex agent can be implemented in Java or XML. Plans of the agent are implemented as java classes derived from foundation plan classes to access the necessary basic services. Other components such as beliefs and goals are implemented as the XML-based ADF (agent definition file). We will describe the detailed implementation procedure of the Jadex agent with the example shown in Fig. 2.

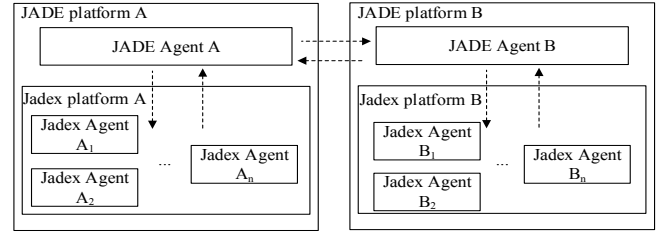


Figure 6. Integrated architecture of the Jadex and JADE platform

A. The implementation of plans

Jadex agent has a plan library, and the plan in the library contains a sequence of actions. All of the actions defined in the plan body are implemented in java. Context conditions and internal event conditions are declared in the plan head (As shown in Fig. 7). The declaration in the plan head (line 1-6) includes the plan name (line 1), plan body (line 2) and the goal that can trigger the plan (line 4). The class *Fire_Extinguish* in the plan body is derived from the foundation class *Plan* (line 7), which enables the derived class to access the basic services provided by infrastructures. In order to achieve the goal *fire extinguishing*, it has to satisfy the sub-goals *electrical fire extinguishing* and *electrical fire extinguishing*. Therefore, it needs to create the goal *electrical fire extinguishing* (line 9), and delegate it to the *electrical fire extinguishing agent* to achieve (line 11). Similarly, it is required to create a goal *electrical fire extinguishing*, and delegate it to the *electrical fire extinguishing agent*.

```

1 <plan name= Extinguish_Fire>
2 <body>new fire_extinguish(<body>
3 <trigger>
4 <goal ref="fire extinguishing"/>
5 </trigger>
6 </plan>
7 public class Fire_Extinguish extends Plan{
8 public void body(){
9 Goal Elec_Fire_Ext = createGoal("electrical fire
extinguishing");
10 dispatch_subgoal(Elec_Fire_Ext)
11 Goal Ord_Fire_Ext= creategoal("ordinary fire
extinguishing");
12 dispatchSubgoal(play); } }
```

Figure 7. The implementation of plans

B. The implementation of beliefs

Jadex agent has a belief base which contains beliefs or belief sets. Note that, the only difference between the beliefs and belief sets is that, the beliefs contain only one fact whereas the belief sets contain multiple facts (As shown in Fig. 8). Beliefs include fire types (1-3), fire extinguisher status (such as the gas pressure, gas concentration, water pressure and the water level) (4-11), and the operating status of the fire monitor (12-14).

C. The implementation of goals

Jadex has four kinds of goals, i.e. the *achieve goal*, *maintain goal*, *query goal* and the *perform goal*. Different kind of goals has a distinct state transition diagram, which leads to

different definitions of goals (As shown in Fig. 9). For the *achieve goal*, it needs to define a *create condition* to describe when a goal is adopted (line 2-4, 10-12). The *achieve condition* denotes when a goal is considered to be achieved (5-7, 13-15). For the *maintain goal*, it needs to define a *maintain condition* specifying how to keep the state of the achieved goal (18-20). For the *perform goal*, it only needs to perform the related plans, which does not care whether the goal is satisfied, thus, it only needs to define a *create condition*.

```

1 <belief name=FireType updatarate=1s>
2   <fact>Monitor.get_fire_type()</fact>
3 </belief>
4 <belief set name=Elc_Ext_Status updatarate=3s>
5   <fact>Elc_Ext_Status.gas_pressure()</fact>
6   <fact>Elc_Ext_Status.concentration()</fact>
7 </belief set>
8 <belief set name=Ord_Ext_Status updatarate=3s>
9   <fact>Ord_Ext_Status.HydraulicPressure</fact>
10  <fact>Ord_Ext_Status.WaterPressure</fact>
11 </belief set>
12 <belief name=MonitorState class=long>
13   <fact>Monitor.current_status()</fact>
14 </belief>

```

Figure 8. The implementation of the belief base

```

1 <achievegoal name="electrical fire extinguishing">
2   <creationcondition>
3     beliefbase.FireType==ElectricalFire
4   </creationcondition>
5   <targetcondition>
6     BeliefBase.ElectricalFireExtinguished
7   </targetcondition>
8 </achievegoal>
9 <achievegoal name="ordinary fire extinguishing">
10  <creationcondition>
11    BeliefBase.FireType==OrdinaryFire
12  </creationcondition>
13  <targetcondition>
14    BeliefBase.OrdinaryFireExtinguished
15  </targetcondition>
16 </achievegoal>
17 <maintaingoal name="fire monitor">
18  <maintaincondition>
19    $beliefbase.MonitorState==on
20  </maintaincondition>
21 </maintaingoal>

```

Figure 9. The implementation of goals

V. CONCLUSION

In this paper, we studied the adaptive software requirements modeling techniques, as well as how to transform requirement models into design models. We also described the implementation process of adaptive software based on the integrated platform of the Jadex and the JADE.

To address the problem that the original Tropos goal model lacks of the ability to specify adaptive software requirements, we extended the model by incorporating context conditions and internal event conditions. Besides, we proposed a model transformation based design method to improve the development efficiency. We extended the BDI model referring to the Tropos model to avoid information loss during the transformation process, and proposed a solution to transform

the Tropos goal model into the BDI model. In the implementation phase, we used a Jadex based implementation technique. For the Jadex supporting ordinary java classes, it only needs to compile just once. Furthermore, Jadex agent also can be implemented with XML-based ADF, which can improve the portability and interoperability of agents.

Future work will focus on how to verify and validate the correctness, consistency and integrality of the generated model. Besides, we will implement the model transformation tool to improve software development efficiency.

REFERENCES

- [1] R. Mizouni, M. A. Matar, Z. A. Mahmoud, S. Alzahmi, and A. Salah, "A framework for context-aware self-adaptive mobile applications SPL," *Expert Systems with Applications*, vol. 41, no. 16, pp. 7549–7564, 2014.
- [2] G. Weiss, K. Becker, B. Kamphausen, A. Radermacher, and S. Gérard, "Model-driven development of self-describing components for self-adaptive distributed embedded systems," In *Software Engineering and Advanced Applications (SEAA)*, 2011 37th EUROMICRO Conference on. IEEE, pp. 477–484, 2011.
- [3] A. van Lamsweerde, "Goal-oriented requirements engineering: a roundtrip from research to practice," in *Requirements Engineering Conference*, 2004. *Proceedings. 12th IEEE International*. IEEE, pp. 4–7, 2004.
- [4] J. N. Mazón, J. Pardillo, and J. Trujillo, "A model-driven goal-oriented requirement engineering approach for data warehouses," in *Advances in Conceptual Modeling—Foundations and Applications*, vol. 4802, J. L. Hainaut, et al., Eds. Berlin: Springer Berlin Heidelberg, 2007, pp. 255–264.
- [5] D. Amyot, S. Ghanavati, J. Horkoff, G. Mussbacher, L. Peyton, and E. Yu, "Evaluating goal models within the goal - oriented requirement language," *International Journal of Intelligent Systems*, vol. 25, no. 8, pp. 841–877, 2010.
- [6] R. Ali, F. Dalpiaz, and P. Giorgini, "A Goal Modeling Framework for Self-contextualizable Software," in *Enterprise, Business-Process and Information Systems Modeling*, vol. 29, T. Halpin, et al., Eds. Berlin: Springer Berlin Heidelberg, 2009, pp. 326–338.
- [7] R. Ali, "Modeling and Reasoning about Contextual Requirements Goal-based Framework," unpublished PhD dissertation, University of Trento, Trento, 2010.
- [8] R. Ali, F. Dalpiaz and P. Giorgini, "A Goal-Based Framework for Contextual Requirements Modeling and Analysis," *Requirement Engineering*, vol. 15, no. 4, pp. 439–458, 2010.
- [9] L. Shen, X. Peng and W. Zhao, "Software Product Line Engineering for Developing Self-Adaptive Systems: Towards the Domain Requirements," In *Computer Software and Applications Conference (COMPSAC)*, 2012 IEEE 36th Annual. IEEE, pp. 289–296, 2012.
- [10] N. Khakpour, S. Jalili, C. Talcott, M. Sirjani, and M. Mousavi, "Formal modeling of evolving self-adaptive systems," *Science of Computer Programming*, vol. 78, no. 1, pp. 3–26, 2012.
- [11] Y. Brun, G. Di Marzo Serugendo, C. Gacek, H. Giese, H. Kienle, M. Litoiu, H. Müller, M. Pezzè, and M. Shaw, "Engineering Self-Adaptive Systems through Feedback Loops," in *Software Engineering for Self-Adaptive Systems*, vol. I, B. H. C. Cheng, R. de Lemos, H. Giese, P. Inverardi, and Jeff Magee, Eds. Berlin: Springer Berlin Heidelberg, 2009, pp. 48–70.
- [12] F. Dalpiaz, "Exploiting Contextual and Social Variability for Software Adaptation," unpublished PhD dissertation, University of Trento, Trento, 2011.
- [13] S. Sardina and L. Padgham, "A BDI agent programming language with failure handling, declarative goals, and planning," *Autonomous Agents and Multi-Agent Systems*, vol. 23, no. 1, pp. 18–70, 2011.